

École Nationale des Sciences Appliquées de Fès

Programmation Objet Java

Pr. Younes Lakhri

PROJET : GESTION D'UNE BIBLIOTHÈQUE

Rapport de projet — Version Spring Boot (JPA / REST / Thymeleaf)

Réalisé par : JAWAD KATTEN

Filière : IDIA

Session : 2025 — 2026

Sommaire	Erreur ! Signet non défini.
1. Introduction	3
2. Cahier des charges	3
3. Concepts et technologies utilisés	4
3.1 Spring Boot.....	4
3.2 JPA et Hibernate	4
3.3 Spring Data JPA (repositories)	4
3.4 API REST et contrôleurs	4
3.5 Thymeleaf	4
3.6 Les exigences objet du TP conservées	4
4. Architecture du projet.....	5
5. Modèle de la base de données	6
5.1 Création de la base sous MySQL.....	6
6. Implémentation.....	6
6.1 Le fichier pom.xml	6
6.2 Le fichier application.properties	6
6.3 Les entités (model)	7
6.4 Les repositories (repository)	7
6.5 Le service (service).....	7
6.6 Les contrôleurs REST (controller).....	8
6.7 La page web Thymeleaf (web)	8
7. Tests et exécution	8
7.1 Lancement de l'application	9
7.2 Test 2 — GET — lister les livres.....	9
7.3 Test 3 — POST — ajouter un livre	9
7.4 Test 4 — PUT — modifier un livre	9
7.5 Test 5 — DELETE — supprimer un livre	9
7.6 Test 6 — Recherche par titre et par auteur	10
7.7 Test 7 — Test des erreurs (livre inexistant)	10
7.8 Test des routes complémentaires	10
7.9 Test de la page Thymeleaf dans le navigateur	10
7.10 Captures d'écran de l'exécution sous Eclipse / STS	10
8. Difficultés rencontrées et solutions	11
9. Conclusion.....	11
Annexe A — Fichiers de configuration.....	12

1. Introduction

Dans le cadre du module Programmation Objet Java, le projet de gestion d'une bibliothèque a d'abord été développé en Java « pur » (JDBC, DAO, Swing). Pour approfondir la maîtrise de la plateforme Java et des frameworks industriels, nous avons réalisé une deuxième version de la même application avec Spring Boot : les mêmes concepts objet (enum, interface, polymorphisme, static) sont réutilisés, mais la persistance est désormais confiée à JPA/Hibernate et les services sont exposés sous la forme d'une API REST consommée par une page Thymeleaf.

Cette deuxième approche reproduit fidèlement la démarche du TP « Gestion des produits pharmacie » (Pr. Youness IDRISSE KHAMLI) : entités JPA, repository Spring Data, service, contrôleur REST et tests avec Postman. Elle illustre le passage d'une couche d'accès manuelle (PreparedStatement) à une couche d'accès générée (JpaRepository), sans changer le modèle de données.

Le présent rapport présente le cahier des charges, les concepts du framework, l'architecture Maven du projet, le modèle relationnel, l'implémentation des entités, des repositories, des services et des contrôleurs, puis les tests d'exécution (console, Postman, navigateur) accompagnés de captures d'écran.

2. Cahier des charges

Le cahier des charges est identique à celui du projet initial ; il exige :

- Ajouter un livre : titre, année de publication, auteur(s), éditeur, catégorie, nombre d'exemplaires ;
- Modifier et supprimer un livre ;
- Afficher la liste des livres avec les noms de leurs auteurs ;
- Rechercher un livre par titre ou par nom d'auteur ;
- Ajouter et gérer les auteurs (nom, prénom, nationalité, date de naissance) ;
- Exposer ces opérations via une API REST (format JSON) et les rendre accessibles dans un navigateur.

Sur le plan technique, la version Spring Boot doit conserver les exigences objet du TP : l'énumération Catégorie et l'interface Archivage (ici une opération REST d'archivage d'un livre). Elle doit utiliser Spring Boot 3, Spring Data JPA, Hibernate, une base de données MySQL et un client REST (Postman).

3. Concepts et technologies utilisés

3.1 Spring Boot

Spring Boot permet de construire une application Java autonome : le serveur Tomcat intégré, l'autoconfiguration et les « starters » (dépendances préconfigurées) éliminent l'essentiel de la configuration manuelle. Une application Spring Boot se lance par un simple main annoté `@SpringBootApplication`.

3.2 JPA et Hibernate

JPA (Jakarta Persistence) est la spécification Java de persistance ; Hibernate en est l'implémentation par défaut dans Spring Boot. Les tables sont mappées sur des classes Java par des annotations (`@Entity`, `@Id`, `@ManyToMany`...) et le framework génère les requêtes SQL. La relation plusieurs-à-plusieurs livre-auteur est modélisée avec `@ManyToMany` et `@JoinTable`, ce qui correspond à la table de liaison `livre_auteur` du modèle JDBC.

3.3 Spring Data JPA (repositories)

Spring Data génère automatiquement les implémentations des interfaces héritant de `JpaRepository`. Les méthodes de recherche sont dérivées du nom de la méthode : `findByTitreContainingIgnoreCase`, `findByAuteursNomContainingIgnoreCase`... Ce mécanisme remplace tout le code SQL de `LivreDAO` et `AuteurDAO`.

3.4 API REST et contrôleurs

L'API REST expose les ressources livres et auteurs. Chaque ressource possède un identifiant et est manipulée par des verbes HTTP : GET (lecture), POST (création), PUT (mise à jour), DELETE (suppression). Les contrôleurs annotés `@RestController` renvoient des objets convertis en JSON ; Postman permet de tester chaque route et de vérifier les codes de statut HTTP (200, 201, 204, 404).

3.5 Thymeleaf

Thymeleaf est le moteur de templates du serveur. Il rend côté serveur une page HTML (`index.html`) servie à `http://localhost:8080`, en affichant dynamiquement la liste des livres dans un navigateur sans client REST.

3.6 Les exigences objet du TP conservées

L'énumération `Categorie` et l'interface `Archivage`, déjà étudiées dans la version JDBC, sont réutilisées : la catégorie est stockée en base avec

@Enumerated(EnumType.STRING) et l'archivage d'un livre est exposé comme opération REST.

4. Architecture du projet

Le projet est un projet Maven généré depuis start.spring.io (ou l'assistant Spring de Spring Tools Suite / Eclipse). Le package principal est ensaf.bibliotheque :

```
bibliotheque-springboot/
├─ pom.xml
├─ src/main/java/ensaf/bibliotheque/
│  └─ BibliothequeApplication.java
│  └─ model/      Categorie.java, Archivage.java, Auteur.java, Livre.java
│  └─ repository/ AuteurRepository.java, LivreRepository.java
│  └─ service/     LivreService.java
│  └─ controller/ LivreController.java, AuteurController.java,
│                  CategorieController.java, ArchivageController.java
│  └─ web/         WebController.java
├─ src/main/resources/
│  └─ application.properties
│  └─ templates/    index.html
└─ src/main/resources/static/
```

Le tableau suivant résume le rôle de chaque couche :

Couche / classe	Rôle
model (Categorie, Archivage, Auteur, Livre)	Entités et types du domaine ; Livre est une entité JPA liée à Auteur par @ManyToMany.
repository (LivreRepository, AuteurRepository)	Interfaces JpaRepository : accès CRUD et recherches dérivées, générés par Spring Data.
service (LivreService)	Logique métier : gestion des livres, des auteurs et de la recherche.
controller (LivreController, AuteurController, ...)	API REST : routes /api/livres, /api/auteurs, /api/categories, /api/archivage.
web (WebController + index.html)	Page Thymeleaf affichant le catalogue dans le navigateur.
BibliothequeApplication	Point d'entrée : @SpringBootApplication, classe main de lancement.

5. Modèle de la base de données

Le modèle relationnel est strictement identique à celui de la version JDBC : deux entités, livre et auteur, reliées par une relation plusieurs-à-plusieurs matérialisée par la table de liaison livre_auteur.

```
auteur (id_auteur PK, nom, prenom, nationalite, date_naissance)
livre (id_livre PK, titre, annee, editeur, categorie, nb_exemplaires)
livre_auteur (id_livre FK, id_auteur FK) PK (id_livre, id_auteur)
```

Avec Hibernate, le schéma peut être généré automatiquement à partir des entités (option ddl-auto). La table livre_auteur est créée par l'annotation @JoinTable. Les données de démonstration sont insérées par un script data.sql (idempotent) au démarrage de l'application.

5.1 Création de la base sous MySQL

La base bibliotheque est créée soit par l'exécution du script SQL, soit automatiquement par Hibernate au premier démarrage (spring.jpa.hibernate.ddl-auto=update).

6. Implémentation

Le projet a été généré via start.spring.io avec les starters : Web, Data JPA, Thymeleaf et le connecteur MySQL. Chaque partie est décrite ci-dessous.

6.1 Le fichier pom.xml

Les dépendances essentielles du projet (Spring Boot 3.2.x, Java 17) :

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency><dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency><dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency><dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>
```

6.2 Le fichier application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/bibliotheque?useUnicode=true&
characterEncoding=utf8
spring.datasource.username=root
spring.datasource.password=****
spring.jpa.hibernate.ddl-auto=update
```

```
spring.jpa.show-sql=true
spring.sql.init.mode=always
```

6.3 Les entités (model)

La classe Livre conserve les exigences objet du TP (enum Categorie, interface Archivage) et les annotations JPA (relation M-N avec Auteur) :

```
@Entity
public class Livre implements Archivage {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String titre;
    private int annee;
    private String editeur;
    private int nbExemplaires;
    @Enumerated(EnumType.STRING)
    private Categorie categorie;
    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(name = "livre_auteur",
        joinColumns = @JoinColumn(name = "id_livre"),
        inverseJoinColumns = @JoinColumn(name = "id_auteur"))
    private List<Auteur> auteurs = new ArrayList<>();

    @Override public void sauvegarder() { System.out.println("Archivé: " +
        titre); }
}
```

6.4 Les repositories (repository)

Spring Data génère les requêtes à partir des noms de méthodes. La recherche par titre et par nom d'auteur remplace les méthodes de LivreDAO de la version JDBC :

```
public interface LivreRepository extends JpaRepository<Livre, Long> {
    List<Livre> findByTitreContainingIgnoreCase(String titre);
    List<Livre> findByAuteursNomContainingIgnoreCase(String nom);
}
```

6.5 Le service (service)

LivreService centralise la logique métier et est injecté dans les contrôleurs par le constructeur :

```
@Service
public class LivreService {
    private final LivreRepository livreRepo;
    private final AuteurRepository auteurRepo;

    public LivreService(LivreRepository livreRepo, AuteurRepository auteurRepo)
    {
        this.livreRepo = livreRepo; this.auteurRepo = auteurRepo;
    }
}
```

```

    public List<Livre> rechercher(String titre, String auteur) { ... }
}

```

6.6 Les contrôleurs REST (controller)

LivreController expose les routes de l'API. Les codes de statut sont gérés par @ResponseStatus et ResponseEntity : 200 (liste), 201 (création), 204 (suppression), 404 (ressource absente).

```

@RestController @RequestMapping("/api/livres")
public class LivreController {
    @GetMapping public List<Livre> lister() { return service.lister(); }
    @GetMapping("/{id}") public ResponseEntity<Livre> trouver(@PathVariable
Long id) { ... }
    @PostMapping public ResponseEntity<Livre> ajouter(@RequestBody Livre livre)
{ ... }
    @PutMapping("/{id}") public ResponseEntity<Livre> modifier(@PathVariable
Long id,
                                                                @RequestBody Livre livre) { ... }
    @DeleteMapping("/{id}") public ResponseEntity<Void> supprimer(@PathVariable
Long id) { ... }
    @GetMapping(params = {"titre", "auteur"})
    public List<Livre> rechercher(@RequestParam String titre, @RequestParam
String auteur) { ... }
}

```

Un contrôleur dédié expose l'archivage : POST /api/archivage/{id} appelle la méthode sauvegarder() de l'interface Archivage, conformément à l'exigence du TP.

6.7 La page web Thymeleaf (web)

WebController sert la page index.html à http://localhost:8080. La liste des livres est affichée dans un tableau avec une boucle th:each, et un formulaire permet d'ajouter un livre depuis le navigateur.

```

@Controller
public class WebController {
    @GetMapping("/")
    public String index(Model model) {
        model.addAttribute("livres", service.lister());
        return "index";
    }
}

```

7. Tests et exécution

L'application se lance en console avec la commande « mvn spring-boot:run » (ou Run As → Spring Boot App dans STS/Eclipse). Le serveur Tomcat démarre par défaut sur le port 8080. Les tests ont été réalisés d'abord avec Postman (API REST), puis dans le navigateur (page Thymeleaf).

7.1 Lancement de l'application

Résultat attendu dans la console Spring Boot :

```
Starting BibliothequeApplication using Java 17 ...
Tomcat started on port 8080 (http)
Hibernate: create table livre (...)
Hibernate: create table auteur (...)
Hibernate: create table livre_auteur (...)
Started BibliothequeApplication in 3.2 seconds
```

7.2 Test 2 — GET — lister les livres

Requête :

```
GET http://localhost:8080/api/livres
```

Réponse attendue :

```
HTTP 200 OK
[ { "id": 1, "titre": "Programmation Java", "annee": 2025,
  "editeur": "Dunod", "nbExemplaires": 5,
  "categorie": "SCIENCE",
  "auteurs": [ { "id": 1, "nom": "Martin", ... } ] }, ... ]
```

7.3 Test 3 — POST — ajouter un livre

Requête :

```
POST http://localhost:8080/api/livres
Content-Type: application/json
{ "titre": "Recettes du Maroc", "annee": 2024,
  "editeur": "La Croisée", "nbExemplaires": 3,
  "categorie": "CUISINE", "auteurs": [ { "id": 1 } ] }
```

Réponse attendue :

```
HTTP 201 Created
{ "id": 4, "titre": "Recettes du Maroc", ... }
```

7.4 Test 4 — PUT — modifier un livre

Requête :

```
PUT http://localhost:8080/api/livres/4
Content-Type: application/json
{ "titre": "Recettes de saison", "annee": 2024, ... }
```

Réponse attendue :

```
HTTP 200 OK
{ "id": 4, "titre": "Recettes de saison", ... }
```

7.5 Test 5 — DELETE — supprimer un livre

Requête :

```
DELETE http://localhost:8080/api/livres/4
```

Réponse attendue :

```
HTTP 204 No Content
```

7.6 Test 6 — Recherche par titre et par auteur

Requête :

```
GET http://localhost:8080/api/livres?titre=Java
auteur=Martin
(ou GET /api/livres?titre=Java puis ?auteur=Martin)
```

Réponse attendue :

```
HTTP 200 OK
[ { "id": 1, "titre": "Programmation Java", ... } ]
[ { "id": 2, "titre": "Python Facile", "auteurs": [ { "nom": "Martin" } ] } ]
```

7.7 Test 7 — Test des erreurs (livre inexistant)

Requête :

```
GET http://localhost:8080/api/livres/999
```

Réponse attendue :

```
HTTP 404 Not Found
{ "message": "Livre non trouvé avec l'id 999" }
```

7.8 Test des routes complémentaires

La liste des catégories (GET /api/categories) renvoie [ROMAN, SCIENCE, CUISINE] et la route d'archivage POST /api/archivage/1 retourne un message confirmant l'appel de la méthode sauvegarder() de l'interface Archivage.

7.9 Test de la page Thymeleaf dans le navigateur

En ouvrant <http://localhost:8080> dans un navigateur, la page index affiche le catalogue complet des livres (titre, année, éditeur, catégorie, exemplaires et auteurs) dans un tableau HTML.

7.10 Captures d'écran de l'exécution sous Eclipse / STS

Les captures suivantes illustrent l'exécution du projet Spring Boot dans l'environnement de développement : structure du projet Maven, éditeur de code, lancement et console.

8. Difficultés rencontrées et solutions

- **Connexion MySQL refusée** — Le service MySQL n'était pas démarré ou le mot de passe était incorrect. Solution : démarrer MySQL et corriger `spring.datasource.username/password` dans `application.properties`.
- **Erreur de mapping Hibernate (relation M-N)** — Oubli du `@JoinTable` ou de la colonne de la clé étrangère. Solution : déclarer explicitement `joinColumns` et `inverseJoinColumns` pour obtenir la table `livre_auteur`.
- **Serialisation JSON en boucle infinie** — La relation `livre ↔ auteur` est bidirectionnelle. Solution : côté entité, ne sérialiser qu'un sens (les auteurs dans le livre) en évitant le getter retour vers les livres dans `Auteur`.
- **Erreur 404 sur /api/livres** — L'application n'était pas démarrée ou le port 8080 était occupé. Solution : vérifier la console de démarrage et changer le port si nécessaire (`server.port` dans `application.properties`).
- **Thymeleaf ne rend pas la page** — Fichier `index.html` absent de `src/main/resources/templates` ou starter manquant. Solution : ajouter `spring-boot-starter-thymeleaf` et placer le template au bon endroit.

9. Conclusion

Cette deuxième version du projet de gestion de bibliothèque nous a permis de découvrir la manière industrielle de développer des applications Java avec Spring Boot. La comparaison avec la version JDBC (projet du cours JAVA) met en évidence les apports du framework : génération automatique des requêtes par Spring Data, mapping objet-relationnel de Hibernate, exposition immédiate des données en JSON via REST, et rendu serveur avec Thymeleaf. Les concepts objet du TP (enum, interface, polymorphisme, static) ont été conservés tels quels, ce qui montre leur indépendance vis-à-vis du framework.

L'application répond à l'ensemble du cahier des charges : CRUD des livres et des auteurs, relation plusieurs-à-plusieurs, recherche par titre et par auteur, API REST testée avec Postman et interface navigateur. En perspective, ce travail pourra être enrichi avec la sécurité (Spring Security), la documentation automatique de l'API (OpenAPI/Swagger) et la gestion des exemplaires empruntés.

Annexe A — Fichiers de configuration

a) pom.xml (dépendances principales) :

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.2.5</version>
</parent>
<dependencies>
  <dependency>spring-boot-starter-web</dependency>
  <dependency>spring-boot-starter-data-jpa</dependency>
  <dependency>spring-boot-starter-thymeleaf</dependency>
  <dependency>com.mysql:mysql-connector-j (runtime)</dependency>
</dependencies>
```

b) application.properties :

```
spring.datasource.url=jdbc:mysql://localhost:3306/bibliotheque?useUnicode=true&
characterEncoding=utf8
spring.datasource.username=root
spring.datasource.password=****
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.sql.init.mode=always
```

c) Script de création de la base (équivalent de init_biblio.sql) :

```
CREATE DATABASE IF NOT EXISTS bibliotheque
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE bibliotheque;

CREATE TABLE IF NOT EXISTS auteur (
  id_auteur      INT AUTO_INCREMENT PRIMARY KEY,
  nom            VARCHAR(50) NOT NULL,
  prenom        VARCHAR(50) NOT NULL,
  nationalite    VARCHAR(50),
  date_naissance DATE
);
CREATE TABLE IF NOT EXISTS livre (
  id_livre       INT AUTO_INCREMENT PRIMARY KEY,
  titre         VARCHAR(100) NOT NULL,
  annee         INT,
  editeur       VARCHAR(50),
  categorie      VARCHAR(20),
  nb_exemplaires INT DEFAULT 1
);
CREATE TABLE IF NOT EXISTS livre_auteur (
  id_livre INT NOT NULL,
  id_auteur INT NOT NULL,
  PRIMARY KEY (id_livre, id_auteur),
  FOREIGN KEY (id_livre) REFERENCES livre(id_livre) ON DELETE CASCADE,
  FOREIGN KEY (id_auteur) REFERENCES auteur(id_auteur) ON DELETE CASCADE
);
```